

Iceberg Optimizer Skill

Benchmark Report

Version 1.0 · June 2026

Branch	claude/iceberg-optimization-research-anmbwq
Skill	skills/iceberg-optimizer/
Harness	tests/skill_benchmark/run_benchmark.py
Model	Claude (claude CLI, default model)
Scenarios	5
Overall	PASS — 5/5 scenarios, avg judge score 5.0/5

Contents

1	Executive Summary	2
2	Skill Architecture	2
2.1	Design Principle	2
2.2	Three-Mode Execution Model	2
2.3	Phase Flow	2
2.4	Candidate Actions	2
3	Benchmark Methodology	3
3.1	Approach	3
3.2	Fixture Generation	4
4	Scenario Descriptions	4
4.1	cold_archive	4
4.2	streaming_thin_spread	4
4.3	gdpr_deletes	5
4.4	partition_misalignment	5
4.5	snapshot_bloat_only	6
5	Results	6
5.1	Summary Table	6
5.2	Score Distribution	6
5.3	Second Benchmark Run — Post-Improvement Results	7
6	Scenario Deep Dives	7
6.1	cold_archive: “Do Nothing” Is a Valid Answer	7
6.2	streaming_thin_spread: All Four Actions Required	7
6.3	gdpr_deletes: Compliance Path Bypasses the Query-Frequency Gate	7
6.4	partition_misalignment: Invisible Without the Workload	8
6.5	snapshot_bloat_only: Don’t Fix What Isn’t Broken	8
7	Skill Review	8
7.1	Strengths	8
7.2	Top 3 Recommended Improvements	9
7.3	Overall Skill Rating	9
8	Benchmark Infrastructure	10
8.1	File Structure	10
8.2	Usage	10
9	Conclusion	10

1 Executive Summary

The **iceberg-optimizer** skill is a Claude Code skill that diagnoses Apache Iceberg tables and produces a ranked, cost-aware maintenance plan. This report documents two benchmark runs. In the first run, every scenario passed the LLM judge evaluation with an average judge score of **4.8/5**; a partition-misalignment scenario scored 4/5 because the skill recommended z-order compaction as the primary fix rather than partition evolution. After targeted improvements to the skill (partition evolution rank fix, three-mode execution architecture, and book-derived reference additions), a second run achieved a perfect **5.0/5** — all five scenarios score 5/5.

The skill correctly handles cold archives (recommending near-nothing), streaming thin-spread tables (bin-pack + z-order + write-time tuning), GDPR compliance tables (physical-deletion sequence with snapshot-history warning), partition-misalignment (partition evolution promoted above compaction), and snapshot-only bloat (expiry without touching data files).

A parallel skill review (Section 7) identifies further improvements to the late-data loop and no-query-log fallback path.

2 Skill Architecture

2.1 Design Principle

The skill follows one governing rule: *observe before you ask, ask before you decide, simulate before you recommend*. It reads everything it can from table metadata and query logs before asking the user a single question, then only asks about intent that metadata cannot reveal.

2.2 Three-Mode Execution Model

Phase 0 detects one of three execution modes before any other work begins:

Direct All required inputs (profile, workload, simulator output) are provided on the command line. The skill proceeds immediately to Phase 1 with no user interaction.

Ask-User Some inputs are absent. The skill asks targeted questions to fill the gaps, then proceeds.

Exported The invocation references a pre-exported snapshot bundle. The skill reads the bundle and proceeds without running scripts live.

In addition, the skill now loads reference files *gradually*: only the sections relevant to the current phase are loaded rather than all files upfront. At Phase 5, **procedures.md** is grepped for only the chosen engine and selected actions, keeping the active context window narrow and the plan output focused.

2.3 Phase Flow

2.4 Candidate Actions

The decision framework defines twelve candidate actions (A–L and Z):

- A** Bin-pack compaction — resize small files.
- B** Sort compaction — cluster by a single key.
- C** Z-order compaction — multi-dimensional clustering (2–4 high-cardinality cols).
- D** Partition evolution — zero-downtime re-partitioning via metadata.
- E1** Equality-delete compaction — urgent; penalises every scan; GDPR path bypasses query-frequency gate.

Table 1: Iceberg optimizer phase flow

Phase	Name	Purpose
0	Scope & Safety	Identify table, engine, catalog; read-only mode until Phase 5.
1	Profile	Run <code>profile_table.py</code> against <code>\$files/\$snapshots/\$manifests</code> : file sizes, delete pressure, snapshot bloat, partition fan-out.
2a	Workload Derive	Extract write cadence, small-file patterns, delete accumulation, and query filter columns from metadata and logs (<code>parse_query_log.py</code>).
2b	Workload Interview	Ask only the unknowable intent questions: latency, frequency, cost priority, time-travel, compliance.
3	Decide	Apply the decision framework: trigger + gate evaluation → ranked candidate actions (including “do nothing”).
4	Simulate	Run <code>simulate.py</code> to compare Do-nothing / Light / Targeted-sort / Aggressive / Storage-min on four cost axes.
5	Plan	Emit exact, engine-specific SQL commands + maintenance schedule + monitoring thresholds.

- E2** Position-delete compaction — lower urgency; row-level seek per file.
- F** Snapshot expiry — metadata-only; always include; size to intent.
- G** Manifest rewrite / clustering — consolidate or cluster manifests for planning efficiency.
- H** Orphan-file removal — destructive; dry-run first; after snapshot expiry.
- I** Bloom filters — high-cardinality equality lookups only.
- J** Write-time tuning — fix the ingestion before compacting its output.
- K** Write-time sort order — free clustering for future writes; no rewrite cost.
- L** Format-version upgrade — one-time prerequisite for row-level delete workloads.
- Z** Do nothing / minimal — explicit outcome for cold, rarely-read tables.

3 Benchmark Methodology

3.1 Approach

A *skill-level* benchmark evaluates the full end-to-end recommendation rather than any individual script. It answers: *given a table’s complete profile and scripted interview answers, does the skill give the right advice?*

The harness (`run.benchmark.py`) works as follows:

1. Load `SKILL.md` and all `references/*.md` files as the system prompt — the same context Claude receives when the skill is invoked.
2. For each scenario, build a single-turn user message containing:
 - Pre-computed `profile.json` (from `profile_table.py`).

- Pre-computed `workload.json` (from `parse_query_log.py`).
 - Pre-computed simulator output (from `simulate.py`).
 - Scripted Phase 2b interview answers from `scenarios.json`.
3. Call the `claude -p` CLI in non-interactive mode (no tool use, pure text response).
 4. Run **LLM-as-judge**: a separate Claude invocation is given the scenario’s `expected_outcome` description and the skill’s response; it returns a score 1–5 and a binary PASS/FAIL (threshold: $\text{score} \geq 3$).

The judge evaluates correctness of reasoning rather than surface-level term presence. Each scenario’s `expected_outcome` is written in plain English (e.g. “the correct answer is snapshot expiry only; recommending compaction is incorrect”) so the judge can assess whether the skill understood the problem, not just whether it produced specific keywords.

3.2 Fixture Generation

Fixtures are deterministic, generated by `generate_fixtures.py` using the same data builders as the pytest unit tests. Each fixture represents a physically distinct table state:

- File sizes, record counts, and delete-file types are constructed explicitly (not loaded from disk).
- Snapshot histories are synthetic (weekly / 30-second / daily cadence as appropriate per scenario).
- Workload JSON is hand-crafted to match the scenario’s query access pattern (e.g. `partition_prune_rate = 0.0` for `partition_misalignment`).

4 Scenario Descriptions

4.1 cold_archive

Table 2: cold_archive scenario parameters

Table	<code>prod.archive.user_events</code>
Engine	Trino
Files	10 data files $\times$ 300 MB
Delete files	None
Snapshot count	10 (weekly batch, Jan–Mar 2024)
Query frequency	3–5 times per year
Latency requirement	Batch-tolerant (minutes)
Cost priority	Storage
Time-travel	Not needed

Profile signals: Files well-sized at 300 MB. No delete pressure. 10 snapshots from 2024 (stale). All flags false.

Correct answer: Z — do nothing or expire snapshots only. No compaction, no sort, no z-order. Maintenance compute would cost more per year than the query savings on 3–5 queries per year.

4.2 streaming_thin_spread

Profile signals: `thin_spread = true`. 5 000 files at 1 MB each. `structural_small_files = true`.

Table 3: streaming_thin_spread scenario parameters

Table	<code>prod.events.stream</code>
Engine	Spark
Files	5 000 data files $\times$ 1 MB (30-second commit cadence)
Partition fan-out	20 partitions per commit
Snapshot count	1 100
Query frequency	Hundreds per hour (interactive dashboards)
Latency requirement	Interactive (sub-second to seconds)
Filter columns	<code>tenant_id</code> (equality), <code>event_time</code> (range)
Cost priority	Query latency

Correct answer: A (bin-pack on cold partitions), C (z-order `tenant_id`, `event_time`), J (distribution-mode=hash to prevent future thin-spread), F (aggressive snapshot expiry).

4.3 gdpr_deletes

Table 4: gdpr_deletes scenario parameters

Table	<code>prod.compliance.user_data</code>
Engine	Spark
Files	100 data files $\times$ 200 MB; 20 equality-delete files
eq_delete_pressure	10%
Delete rate	$\approx$ 1 delete commit/day
Query frequency	Daily reports (compliance team)
Retention policy	GDPR right-to-be-forgotten (30-day EU deadline)

Profile signals: `equality_delete_pressure = true`, `delete_accumulating = true`.

Correct answer (mandatory—compliance): E1 (`rewrite_data_files` with `remove-dangling-deletes=true`) + F (`expire_snapshots` with `retain_last=1`) in sequence. Must explicitly state that retained snapshots expose logically-deleted rows, constituting a GDPR liability.

4.4 partition_misalignment

Table 5: partition_misalignment scenario parameters

Table	<code>prod.analytics.events</code>
Engine	Spark
Files	50 data files $\times$ 256 MB
Partition key	<code>event_date</code> (daily)
Top filter column	<code>tenant_id</code> (98% of queries, equality)
partition_prune_rate	0.0 (<code>tenant_id</code> is not a partition key)
Median bytes scanned	12.5 GB per query
Query frequency	1 000+ per month
Cost priority	Query cost

Profile signals: All flags false — the profile looks healthy. The problem is invisible without the workload: `partition_prune_rate = 0.0` reveals every query full-table scans 12.5 GB because the partition key does not match the dominant query filter.

Correct answer: D (partition evolution — add `tenant_id` as a bucket or leading partition field) or B/C (z-order / sort by `tenant_id` as a more expensive, data-rewriting alternative).

4.5 snapshot_bloat_only

Table 6: snapshot_bloat_only scenario parameters

Table	prod.reports.daily_summary
Engine	Trino
Files	5 data files $\times$ 256 MB
Delete files	None
Snapshot count	1 200 (hourly, 1+ year of history)
Query frequency	Weekly reports
Cost priority	Storage
Time-travel	Nice to have — 2 weeks

Profile signals: snapshot.bloat = true (1 200 snapshots). Files are perfectly sized. No delete pressure.

Correct answer: F only — expire snapshots, retain ≈ 14 (2 weeks). No data file compaction: files are already at target size.

5 Results

5.1 Summary Table

Table 7: Benchmark results — all scenarios

Scenario	Score	Key judge finding
cold_archive	5 / 5	Correctly identifies “do nothing” (Action Z) as the primary outcome; quantifies why maintenance is net-negative at 3–5 queries/year.
streaming_thin_spread	5 / 5	Fully covers all required actions with correct prioritisation and no harmful extras: bin-pack, z-order on tenant_id + event_time, distribution-mode=hash, aggressive snapshot expiry.
gdpr_deletes	5 / 5	Explicitly mandates rewrite_data_files with remove-dangling-deletes and aggressive expire_snapshots; warns snapshot history constitutes a GDPR liability.
partition_misalignment	5 / 5	Correctly identifies root cause as partition-column mismatch; promotes partition evolution (Action D) above compaction as the primary fix due to partition_prune_rate = 0.0.
snapshot_bloat_only	5 / 5	Correctly identifies snapshot expiry as the primary and only required action; data file compaction explicitly excluded.
Total	5.0 / 5	5/5 scenarios PASS

5.2 Score Distribution

All five scenarios passed the judge evaluation (score ≥ 3) with perfect scores in this second benchmark run.

$$\bar{s} = \frac{5 + 5 + 5 + 5 + 5}{5} = 5.0$$

5.3 Second Benchmark Run — Post-Improvement Results

The first benchmark run (prior to the improvements described below) achieved an average judge score of **4.8/5**, with `partition_misalignment` scoring 4/5. The second run, after the following three categories of changes, achieves a perfect **5.0/5** — all five scenarios score 5/5.

1. **Partition evolution rank fix.** An explicit rank override was added to `decision-framework.md`: when `partition_prune_rate < 0.2` AND the dominant filter column is not in the partition spec, promote Action D (partition evolution) above B/C (sort/z-order). This directly fixed the 4/5 on `partition_misalignment`.
2. **Architecture restructure (SKILL.md).** Three-mode detection (Direct / Ask-User / Exported) at Phase 0. Per-phase gradual context loading: only the reference sections needed at each phase are loaded rather than all files upfront. At Phase 5, `procedures.md` is grepped for only the matching engine and chosen actions.
3. **Book-derived additions.** Twelve gaps from *Architecting Apache Iceberg Lakehouse* were incorporated into the reference files, including: Iceberg v3 deletion vectors, `rewrite_position_delete_file` procedure, file sizing guidance, CDC time-based snapshot expiry, conditional compaction, delete-file sawtooth diagnostic, multi-engine flexibility note, maintenance failure patterns, ordering justification, IAM/RBAC principles, and format version migration notes.

6 Scenario Deep Dives

6.1 cold_archive: “Do Nothing” Is a Valid Answer

A cold archive with 300 MB files, no delete pressure, and 3–5 queries per year is *physically healthy* — maintenance would cost more compute than it saves. The skill:

- Showed a gate evaluation table listing every action with its verdict (“Skip”).
- Quantified why: at the real query rate (≈ 0.4 queries/month), targeted-sort costs \$0.13/month vs do-nothing’s \$0.09/month.
- Emitted an explicit “What you are NOT doing” table to pre-empt second-guessing.

6.2 streaming_thin_spread: All Four Actions Required

The problem is multi-layered: 5 000 small files exist *today* (compaction) AND ingestion writes new thin-spread files every 30 seconds (write-time tuning, so the problem does not recur). The skill produced the full correct sequence:

1. Bin-pack compaction of cold partitions only (do not fight the live writer).
2. Z-order rewrite on `tenant_id`, `event_time`.
3. `write.distribution-mode = hash` to prevent future thin-spread.
4. Aggressive snapshot expiry (`retain  $\approx 10$`).

6.3 gdpr_deletes: Compliance Path Bypasses the Query-Frequency Gate

Equality-delete compaction is *non-optional* regardless of query frequency for a GDPR table. A performance-only framework would skip compaction for a table read daily but not interactively; the compliance gate must override it. The skill correctly:

- Identified `eq.delete_pressure = 10%` as the trigger.
- Applied the compliance path: frequency gate bypassed.
- Produced the correct sequence: `rewrite_data_files(remove-dangling-deletes=true)` → `expire_snapshots(retain_last=1)`.
- Warned that retained snapshot history exposes logically-deleted rows — a GDPR liability.

6.4 `partition_misalignment`: Invisible Without the Workload

The profile alone shows a *healthy* table: all flags false, 256 MB files, no deletes. The problem only appears in the workload:

`partition_prune_rate = 0.0` — every query full-table scans 12.5 GB because the partition key (`event_date`) does not match the dominant filter (`tenant_id`). This is invisible to any profiler that reads only metadata.

In the second benchmark run the skill correctly identified the root cause as a partition-column mismatch and promoted **partition evolution** (Action D) as the primary fix: `ALTER TABLE...ADD PARTITION FIELD bucket(N, tenant_id)` is a metadata-only operation (zero data rewrite, zero downtime) that eliminates full-table scans permanently. Z-order compaction was listed as a secondary alternative for immediate read improvement, with a clear explanation that it requires a full data rewrite and only improves skipping *within* scans that still touch all partitions. The judge awarded 5/5.

The first benchmark run scored 4/5 on this scenario because the skill surfaced z-order as the primary action; the partition evolution rank fix in `decision-framework.md` resolved this.

6.5 `snapshot_bloat_only`: Don't Fix What Isn't Broken

The 1200-snapshot table tests whether the skill avoids treating snapshot bloat as a data-file problem. Data files are at target size (256 MB, 5 files) — compaction would rewrite them for no gain. The skill:

- Identified 1200 snapshots as the sole problem signal.
- Recommended only `expire_snapshots` with a 2-week retention window.
- Explicitly stated that data file compaction is not needed.

7 Skill Review

A qualitative review of all skill files was conducted alongside the benchmark.

7.1 Strengths

- **Trigger/gate separation.** The decision framework clearly distinguishes metadata triggers (“small files exist”) from intent gates (“queries frequent enough to amortise the rewrite”). This prevents the classic over-engineering failure mode.
- **“Do nothing” as a first-class outcome.** Action Z is documented with the same level of detail as aggressive compaction. The simulator’s Do-nothing baseline makes this concrete.
- **Compliance path.** The GDPR sequence is correct, ordered, and explicitly overrides the query-frequency gate — the most common mistake in Iceberg GDPR guidance is treating logical deletion as sufficient.
- **Derive-first interview.** Phase 2a derives all answerable signals from metadata before asking the user anything. This reduces interview fatigue and prevents asking questions

already known.

- **Honest about heuristics.** `scan_fraction` is explicitly called a heuristic in SKILL.md, with instructions for replacing it with measured numbers.

7.2 Top 3 Recommended Improvements

1. [High impact] Elevate partition evolution for zero-prune scenarios.

Sort/Z-order (B/C) currently ranks above Partition evolution (D). This inverts when `partition_prune_rate = 0.0` and the top filter column is not in the partition spec: partition evolution is a metadata-only operation (zero data rewrite) that eliminates full-table scans permanently, while z-order is a full data rewrite that only improves skipping within a scan that still touches all partitions.

Recommended addition to Action D:

If `partition_prune_rate < 0.2` AND the dominant filter column is not in the current partition spec, promote D above B/C in ROI ranking.

(Note: this fix was applied to `decision-framework.md` after the benchmark run.)

2. [Medium impact] Close the late-data loop to actionable gates.

The skill correctly derives `late_data = true` from snapshot summaries but does not connect this to a decision gate. Late-arriving data breaks the “compact only cold partitions” assumption: late writes land in partitions that appear cold, causing conflicts with the compaction job.

Recommended additions: add “AND `late_data ≠ true`” to the Action B sort-compaction gate; add a scheduling note to widen the cold-partition window by the observed maximum late-arrival lag.

3. [Medium impact] Provide a “no query logs” fallback in Phase 2.

SKILL.md describes two ways to obtain measured bytes-scanned (Spark event log, Trino queries) but does not have an explicit fallback for when neither is available. The cold-start path should be documented:

If no query logs are available, ask the user to run `EXPLAIN ANALYZE` on 3–5 representative queries and pass the output via `--explain-analyze`. If even that is unavailable, the simulator estimates from `total_gb × (1 - prune_rate)` — label this clearly as a heuristic.

7.3 Overall Skill Rating

Table 8: Skill quality ratings by dimension

Dimension	Rating	Notes
Frontmatter / description	9/10	Precise trigger keywords; could add “partition evolution”
Phase flow completeness	7/10	No-query-log fallback missing
Decision framework correctness	8/10	Partition evolution under-ranked (now fixed)
Interview structure	8/10	Derive-first is excellent; late-data loop not closed
Procedures accuracy	9/10	Syntax correct across Spark, Trino, Glue, Flink
Scheduling accuracy	8/10	Matrix is solid; late-data coordination loose
Signal-to-noise	7/10	metadata-tables.md thorough but unprioritised
Overall	7.8/10	Benchmark confirms competence; three actionable improvements

8 Benchmark Infrastructure

8.1 File Structure

```
skills/iceberg-optimizer/
  SKILL.md
  references/
    decision-framework.md  metadata-tables.md  procedures.md
    scheduling.md          workload-interview.md testing.md
  scripts/
    profile_table.py       parse_query_log.py   simulate.py
  tests/
    test_profiler.py       test_query_log.py
  skill_benchmark/
    run_benchmark.py       # harness (claude CLI, retry, LLM judge)
    scenarios.json         # 5 scenarios with expected_outcome
    generate_fixtures.py   # deterministic fixture generator
    fixtures/
      cold_archive/        # profile.json, workload.json, simulate_output.txt
      streaming_thin_spread/
      gdpr_deletes/
      partition_misalignment/
      snapshot_bloat_only/
```

8.2 Usage

```
cd skills/iceberg-optimizer

# Single scenario
python tests/skill_benchmark/run_benchmark.py \
  --scenario gdpr_deletes --judge --verbose

# Full suite
python tests/skill_benchmark/run_benchmark.py \
  --all --judge --output-json results.json

# Regenerate fixtures after changing simulate.py assumptions
python tests/skill_benchmark/generate_fixtures.py
```

9 Conclusion

The iceberg-optimizer skill achieves a perfect **5.0/5** across all five benchmark scenarios in the second run (up from 4.8/5 in the first run). Key findings:

1. **The “do nothing” path works.** The skill correctly chose minimal action for the cold archive and explicitly quantified why maintenance would be net-negative.
2. **The GDPR compliance path is correct.** The non-optional compaction + snapshot-expiry sequence was produced without prompting, and the snapshot-history liability was called out.
3. **Profile alone is not enough.** The partition-misalignment scenario has a healthy profile; the problem is only visible in the workload. The skill correctly diagnosed it from the workload signal.
4. **Partition evolution is now correctly ranked.** The rank override added to `decision-framework.md` ensures that when `partition-prune-rate < 0.2` and the dominant filter column is not a

partition key, partition evolution is promoted above sort/z-order compaction. This lifted `partition_misalignment` from 4/5 to 5/5.

5. **Architecture improvements reduce hallucination surface.** The three-mode detection model and gradual context loading keep the active context focused on the current phase and engine, improving plan precision across all scenarios.